

Improving Cloud Security Posture Through Normalized Risk Scoring of Infrastructure-as-Code Security Tools

Alexis Reedy
Information Assurance Student
University of North Florida
Jacksonville, USA

Abstract—Cloud misconfigurations remain one of the most significant security risks in modern cloud computing environments, especially within Infrastructure-as-Code (IaC) applications where configuration errors are replicated across environments when deployed at a large scale [1], [10]. IaC security scanning tools such as Checkov and Terrascan provide automated detection of these misconfigurations. However, they both differ significantly in output structure, rule classification, and severity definitions [6], [7]. These inconsistencies prevent direct comparison between tools by limiting the ability to prioritize vulnerabilities consistently shown within DevSecOps pipelines.

This paper presents a normalization framework that processes security findings from multiple IaC scanning tools and converts them into a standardized internal representation. This system takes terraform configurations, reads structured JSON output from Checkov and Terrascan, and applies a normalization layer that adds severity values and rule metadata. Findings are preserved on a separate tool basis to assist in comparative analysis rather than full aggregation.

This proposed approach is grounded in established cloud risk management frameworks such as OCTAVE, COBIT, and CORAS, which emphasize structured classification and prioritization of security risks based on confidentiality, integrity, and availability (CIA) model [1], [2], [3], [12].

Keywords—Infrastructure as Code (IaC), cloud security, DevSecOps, static analysis, misconfiguration detection, risk normalization, Checkov, Terrascan.

I. INTRODUCTION

Cloud computing has become a core foundational component of software infrastructure today due to the scalability, cost effectiveness, and the support for automated deployment pipelines [11]. Infrastructure-as-Code (IaC) has expanded further on this example and emerged as a standard approach for processing and managing cloud environments through readable configuration files, such as Terraform, for the computers [8].

While IaC provides automation and reproducibility, it also introduces significant security risks such as cloud misconfigurations. These misconfigurations, which refer to incomplete security configurations in cloud services or storage

resources that can result in unintended data exposure or increased vulnerability. These IaC environments by definition can be used repeatedly across many environments leading to widespread vulnerabilities. Common examples of these would be publicly exposed storage buckets, insecure identity and access management (IAM) policies, missing encryption configurations, and weak network access rules [13].

To mitigate these risks, organizations will sometimes rely on static analysis tools such as Checkov and Terrascan that evaluate IaC configuration files against known security misconfigurations before publishing them to the general public [6], [7]. These tools can integrate into DevSecOps pipelines and provide early detection for security issues to limit the number of security risks that go undetected. However, a key limitation is that each tool produces outputs in different formats, severity definitions, and their rule classification systems.

This lack of standardization makes it difficult to compare findings or prioritize vulnerabilities across all tools. As a result, security teams must continuously assess and evaluate inconsistent outputs which will reduce efficiency in automated workflows.

Existing cloud security risk management frameworks such as OCTAVE, COBIT 5, and CORAS provide structured methodologies for identifying and prioritizing risks based on confidentiality, integrity, and availability principles [1], [2], [3], [12]. However, these frameworks are not designed to process diverse outputs generated by automated IaC security scanning tools.

In addition to structural inconsistencies, existing IaC security tools also differ in their detection scope and prioritization strategies, which further complicates their practical use in DevSecOps environments. Some tools emphasize broader rule coverage, such as Checkov, while others focus on specific categories such as identity management or network configurations. As a result, organizations that utilize multiple tools may encounter conflicting findings without a clear method for interpreting their importance. This fragmentation highlights the need for a systematic approach that can standardize how findings are represented and evaluated while preserving the context provided by each tool.

This gap motivates the need for a normalization layer that standardizes security findings into a unified risk representation

model. As such, this paper addresses the following research question: How can diverse Infrastructure-as-Code security scan outputs be standardized into a unified representation to enable consistent cross tool vulnerability analysis?

II. RELATED WORK

A. Cloud Risk Management Frameworks

Risk management in cloud security has been widely studied in both academic and industry contexts. Zhang et al. proposes a structured framework based on traditional security principles and the CIA triad, focusing on identification and mitigation of cloud threats [1]. Kure et al. expands on this work by introducing a predictive risk management framework that includes continuous monitoring and versatile mitigation strategies [2]. Tsaregorodtsev et al. further expands on measurable methods for cloud risk estimation, allowing measurable comparisons across systems [3]. A complete review by Ali et al. highlights inconsistencies across current existing frameworks and the lack of standardized risk evaluation approaches [4].

These frameworks typically follow a structured lifecycle consisting of asset identification, threat modeling, risk assessment, and mitigation planning. While effective at a strategic level, they are primarily designed for manual or semi-automated processes and do not directly integrate with modern DevSecOps pipelines.

While these frameworks operate at a theoretical level, they do not address automated IaC security outputs.

B. Cloud Misconfiguration Risks

Cloud misconfigurations are widely recognized as a leading cause of security breaches. James Guffey and Yanyan Li demonstrate that misconfigured services frequently lead to enterprise size data exposure [11]. This backs up Ede et al. by highlighting challenges in identifying identity and access management misconfigurations at such a large size [12]. Earlier works from K. Wood and E. Pereira associate these issues to system complexity and lack of standardized configuration practices [10].

In addition, misconfigurations are often the result of human error, default insecure settings, or insufficient understanding of cloud service configurations. The increasing complexity of cloud environments further amplifies these risks, particularly in large scale deployments where configuration errors can generate rapidly.

While these studies focus on detection and impact, they do not address how findings from multiple tools can be unified to help with consistency of data outputs.

C. Automated and AI-Based Detection

Recent research explores automated approaches to cloud security analysis to improve efficiency. Saxena et al. puts forward AI driven models to predict cloud threats [13]. While Mostafa Anour Ghorab and Mohamed Aymen Saied investigate Kubernetes misconfiguration detection using large language models [17]. Mahesh Kumar Goyal and Rahul Chaturvedi

explore further augmented retrieval approaches for identifying misconfigurations [19].

These approaches aim to reduce manual analysis and improve detection accuracy by leveraging machine learning and intelligent pattern recognition. However, the outputs generated by these systems often vary significantly in format, terminology, and confidence metrics.

A commonality in these methods is that they often produce outputs that are difficult to standardize across multiple tools, reinforcing the need for more normalization frameworks.

D. Multi-Cloud Security Complexity

Multi-cloud environments introduce extra challenges due to inconsistent security policies across platforms. Essien et al. propose structured vulnerability management procedures for multi cloud systems [15]. On the other hand, Mahesh Sharma highlights increased misconfiguration risks due to inconsistent management models [14]. Calvin Nobles further identifies human factors as a contributing cause of the ever-growing misconfiguration errors [18].

The diversity of cloud service providers and their differing security models create challenges in maintaining consistent policy enforcement and visibility. This division complicates security analysis, particularly when organizations rely on multiple tools and platforms simultaneously.

All these studies highlight the complexity of these cloud environments but do little to provide frameworks for unifying security findings across tools.

III. PROPOSED APPROACH

The proposed system introduces a pipelines-based architecture designed to normalize and unify IaC security findings from multiple static analysis tools converting their outputs into a unified internal representation. The pipeline consists of parsing, normalization, and analysis stages.

From a system design perspective, the pipeline is structured to separate tool specific processing from generalized analysis logic. This flexible approach ensures that each stage of the pipeline operates independently, allowing for flexibility in handling diverse data sources. By isolating parsing, normalization, and analysis into distinct components, the system improves maintainability and enables future integration of additional IaC scanning tools without significant architectural changes.

The system begins with Terraform configuration files as input. These files are then passed through multiple static analysis tools, specifically Checkov and Terrascan [8]. Each tool independently analyzes the configuration files and produces structured JSON outputs containing rule violations, severity classifications, resource identifiers, and remediation suggestions.

Due to the structural differences in JSON outputs generated by each scanning tool, a mechanism is required to correctly identify and direct inputs to the appropriate processing logic. This is necessary because Checkov and Terrascan produce

findings using different schemas, naming conventions, and field structures.

Next the system starts in a class called “Dispatcher”, this method helps the system determine what kind of JSON file has been input into the system. The structure of the JSON file Next, the system uses a class called Dispatcher to determine which kind of JSON file has been input varies greatly. Based on what is read in the file, this method will tell the system whether it needs to use the Checkov parser or the Terrascan parser.

The next step is responsible for parsing tool specific JSON files and converting them into objects in code and storing them for a summary output later. This step preserves all relevant data, such as the affected resource and any severity information, while also separating the outputs from tool specific dependencies.

This transformation into a unified object model abstracts away tool specific implementation details while preserving vital security information. As a result, dependent components can operate on a consistent data structure without requiring knowledge of the original tool format, simplifying further processing and analysis.

Each detected issue is represented as a Finding object, which serves as the core data structure within the system. A Finding encapsulates attributes such as the affected resource, severity level, rule identifier, descriptive message, and source tool. This unified representation allows the system to standardize diverse outputs while maintaining traceability to the original scanner. Additionally, severity scores are assigned or supplemented when necessary to ensure consistency across findings. These objects store what resource is affected, any kind of severity information, adds a severity score, and has a resource link to help fix the problem. These outputs are all combined parts of each scanning tool. Terrascan labels each finding HIGH, MEDIUM, or LOW but provides no numeric risk score. Checkov's output does not include severity without a platform membership. The RiskNormalizer therefore assigns severity to Checkov findings using a classification of rules into HIGH, MEDIUM, and LOW, and converts all severities to a common numeric scale.

Since each issue is represented by a findings object:

$$F = (\text{rule}, \text{severity}, \text{resource}, \text{description}, \text{source})$$

This formula represents a finding object to guarantee consistent representation across tools.

This formal representation ensures that all findings, regardless of their origin, match to a consistent schema. It enables uniform processing across following stages of the pipeline and serves as the foundation for normalization and analysis operations.

The normalization stage represents the core contribution of the proposed system. In this phase, findings are transformed to ensure consistency across tool outputs by applying a set of standardized mapping and alignment operations. These include converting severity labels into a unified ordinal scale (e.g., LOW

= 1, MEDIUM = 2, HIGH = 3), harmonizing rule identifiers where possible, and standardizing resource naming conventions. Also, when severity information is incomplete or absent, the system applies a basic risk assignment to ensure all findings can be comparatively evaluated.

This design aligns with structured risk classifications principles defined in OCTAVE and CORAS frameworks [1], [3], [12], which emphasize consistent categorization of risk attributes for reliable prioritization.

Each normalized finding may be assigned a simplified risk score based on available attributes such as severity level and resource context. This scoring mechanism is intentionally lightweight and is applied primarily when tools do not provide sufficient prioritization information. The goal is not to replace existing risk models, but to ensure that all findings can be evaluated within a consistent comparative framework.

This scoring model helps to produce consistent ranking across tools, ensuring that equivalent vulnerabilities are assigned comparable risk values regardless of origin. This aligns with established cloud risk quantitation research [14].

The final part of this approach is the combination step where normalized findings are merged into a unified summary. This summary produces the total number of misconfigurations found by each tool and reports what each one of them is with the relevant information out of the findings object. This allows users to compare the outputs of each tool with the total amount flagged by each tool as well as scan the outputs for any duplicates to show where commonalities are between them.

It is important to note that findings are not deduplicated or fully merged across tools. Instead, outputs are preserved separately to maintain transparency and traceability to the original source. This design decision allows for clearer comparison of tool behavior, including differences in detection coverage and rule interpretation. By prioritizing consistency in representation over full aggregation, the system provides a practical foundation for analyzing heterogeneous IaC security outputs.

IV. EVALUATION RESULTS

The system was evaluated using 15 Terraform configuration files containing intentionally introduced cloud misconfigurations. Each file was analyzed using Checkov and Terrascan independently, and outputs were processed through the normalization pipeline.

The evaluation was designed to validate each stage of the pipeline independently, including the Dispatcher, parsing components, normalization logic, and final aggregation output. By analyzing how each component contributed to the overall processing workflow, the system's reliability in handling diverse IaC tool outputs could be assessed in a structured manner.

The evaluation focused on validating the correctness of each component within the pipeline, including input routing, parsing accuracy, normalization consistency, and overall output

interpretability. This component-level evaluation ensures that the system functions reliably across diverse tool outputs.

The evaluation also examined how effectively the pipeline handled variations in input complexity and misconfiguration types. By testing multiple configuration files with differing structures and security issues, the system’s behavior across files with different structures and misconfiguration types was assessed. This suggests the approach can handle varied inputs, though a larger evaluation is needed across different IaC configurations.

A. Dispatcher and Parsing Accuracy

Through every round of testing the dispatcher was able to correctly sort the files into the correct parser. Sorting between the Terrascan parser and the Checkov parser based on file structure and terminology in the file helped make the dispatcher 100% correct. This consistent classification demonstrates that the Dispatcher can effectively handle structural variations in JSON outputs, ensuring that tool-specific formats are processed by the appropriate parsing logic without errors propagating into later stages.

Additionally, the parsing stage successfully converted all tool-specific outputs into a unified internal representation using Finding objects. This ensured that essential attributes such as severity, resource identifiers, and rule descriptions were preserved while eliminating dependencies on the original tool format.

In addition to component-level validation, the system was evaluated as an end-to-end pipeline. For each Terraform configuration file, outputs from both Checkov and Terrascan were successfully processed through all pipeline stages without failure. This confirms that the integration between scanning, parsing, normalization, and analysis components functions cohesively under typical usage conditions.

This end-to-end validation confirms that no data loss or structural inconsistencies occurred during transitions between pipeline stages in the tested files. Each component successfully passed processed data to the next stage without requiring manual intervention, demonstrating the reliability of the system’s modular architecture.

B. Normalization Consistency

Similarly, the RiskNormalizer component consistently standardized severity values across all findings. This ensured that outputs from both tools were represented using a unified scale, improving readability and enabling structured comparison. All findings were successfully organized from highest to lowest priority based on normalized severity values.

This normalization process was particularly important for handling discrepancies between tools, as Checkov does not provide numerical risk scores while Terrascan provides categorical severity labels. By converting all outputs into a consistent ordinal scale, the system enabled direct comparison across previously incompatible formats.

Without this normalization step, analysts would be required to manually interpret tool specific severity labels and output formats, increase analysis effort and the likelihood of

misinterpretation. The automated standardization provided by the system reduces this complexity and enables more efficient vulnerability prioritization across multiple tools.

The unified Finding object representation was also validated during evaluation. All parsed outputs were successfully converted into this schema, ensuring that attributes such as severity, resource identifiers, and rule descriptions were consistently preserved. This confirms that the abstraction layer effectively eliminates tool-specific dependencies while maintaining essential security context.

This consistent representation also simplifies following processing, as all findings can be handled uniformly regardless of their origin. As a result, the system avoids repeated logic for handling tool-specific formats and improves overall maintainability and scalability.

C. Tool Output Comparison

A key objective of this evaluation was to analyze differences in detection behavior between Checkov and Terrascan. This comparison provides insight into how each tool contributes to overall security analysis within the pipeline.

Checkov consistently produced a higher number of findings per configuration compared to Terrascan. On average, Checkov identified approximately 7 issues per file, while Terrascan identified approximately 4 issues.

This disparity suggests that different tools prioritize different categories of misconfigurations, which can influence how security risks are perceived. A higher number of findings does not necessarily indicate better detection, but rather reflects differences in rule granularity and policy definitions.

This difference highlights variation in rule coverage and detection strategies between the two tools, with Checkov favoring broader policy enforcement and Terrascan focusing on specific infrastructure level misconfigurations.

Checkov primarily focused on Amazon S3 security misconfigurations, including public access control, encryption enforcement, logging configuration, and lifecycle policies. Terrascan, in contrast, identified issues across EC2 instance metadata configurations, security group ingress rules, and storage versioning policies.

These differences highlight an important insight: no single tool provides complete coverage of cloud misconfigurations. Instead, each tool contributes a partial view based on its rule set and detection priorities. This reinforces the necessity of multi-tool analysis, as relying on a single scanner may result in overlooked vulnerabilities. The proposed normalization approach enables these complementary perspectives to be analyzed together in a consistent manner.

From a security operations perspective, this combined analysis approach allows teams to gain a more comprehensive understanding of their cloud security posture. By aggregating insights from multiple tools, organizations can reduce blind spots and improve overall risk awareness.

These distinctions also demonstrate that each tool emphasizes different areas of cloud security, which can lead to incomplete coverage if only a single tool is used in isolation. The

differences reflect known variability in static analysis coverage and rule design across IaC security tools [10], [13].

The higher number of findings produced by Checkov suggests broader rule coverage, while the smaller number of findings from Terrascan reflects a more targeted detection scope focused on specific infrastructure components.

D. Example Output Analysis

Check ID: AC_AWS_0231

Issue: unrestrictedIngressAccess

Severity: HIGH

Risk Score: 0

Resource: bad_sg

~~~~ SUMMARY ~~~~

Total Issues: 4

HIGH: 3

MEDIUM: 1

LOW: 0

UNKNOWN: 0

This example illustrates the structured output format produced by Terrascan, as well as the limited distribution of severity levels. When compared to Checkov outputs, it becomes evident that severity classifications and detection focus vary significantly between tools.

This variation further reinforces the need for normalization, as differences in reporting formats and severity distributions can otherwise obscure the true prioritization of security issues.

E. Severity Comparison and Limitations

Both tools classify severity, but their classification systems are not directly comparable. Terrascan distributes findings across HIGH, MEDIUM, and LOW categories, while Checkov produces fewer LOW severity findings and a narrower distribution overall. This discrepancy highlights a key limitation in multi-tool environments, where inconsistent severity definitions make direct prioritization difficult without additional normalization.

Additionally, the evaluation dataset was limited to a relatively small number of intentionally misconfigured files. While this controlled setup allows for consistent testing, it may not fully capture the complexity and scale of real-world cloud environments. Future work could include larger datasets and production level configurations to further validate the system's effectiveness.

F. Normalization Impact

None of the findings were merged or deduplicated in order to preserve tool-specific outputs and maintain traceability to the original source. This design enables clearer evaluation of differences in detection strategies and supports direct comparison between tools.

After applying the normalization all findings were mapped into a unified schema and converted into a consistent risk scale.

This enabled direct comparison of previously incompatible outputs. As a result, findings from both tools could be evaluated within the same prioritization framework, improving the overall interpretability of the results.

Overall, the normalization process serves as a critical enabler for multi-tool security analysis by transforming fragmented outputs into a cohesive and comparable format. This allows security practitioners to focus on risk prioritization rather than data interpretation, improving both efficiency and decision-making in cloud security workflows.

Normalization improved interpretability by enabling unified ranking of vulnerabilities across tools and reducing inconsistencies caused by differing severity representations [9], [15].

V. CONCLUSION AND FUTURE WORK

This paper presented a modular framework for processing and analyzing Infrastructure-as-Code security findings from multiple static scanning tools. The proposed approach addresses a fundamental limitation in current DevSecOps workflows: inconsistent output structures and severity classifications across tools.

Evaluation results demonstrated that the system effectively handles diverse outputs while also preserving the specific tool findings. Differences in detection coverage between tools highlight the importance of comparative analysis between IaC security workflows.

By transforming diverse tool outputs into a standardized schema and applying a unified risk scoring model, the proposed system improves comparability and supports more reliable security decision-making.

Future work will focus on extending this framework to additional IaC security tools such as, tfsec and KICS, increasing coverage across cloud environments [6], [9]. Additional enhancements may include machine-learning based risk scoring models that incorporate historical vulnerability data and exploitability metrics [14]. Furthermore, integration into CI/CD pipelines would enable real-time security feedback during infrastructure development and deployment stages [16].

REFERENCES

- [1] Zhang, Xuan, Nattapong Wuwong, Hao Li, and Xuejie Zhang. "Information security risk management framework for the cloud computing environments." In 2010 10th IEEE international conference on computer and information technology, pp. 1328-1334. IEEE, 2010.
- [2] H. I. Kure, S. Islam, and H. Mouratidis, "An integrated cyber security risk management framework and risk prediction for the critical infrastructure protection," *Neural Computing and Applications*, vol. 34, no. 18, pp. 15241-15271, 2022.
- [3] A. V. Tsaregorodtsev, O. Ja Kravets, O. N. Choporov, and A. N. Zelenina, "Information security risk estimation for cloud infrastructure," *International Journal on Information Technologies & Security*, vol. 10, no. 4, 2018.
- [4] T. Ali, M. Al-Khalidi, and R. Al-Zaidi, "Information security risk assessment methods in cloud computing: Comprehensive review," *Journal of Computer Information Systems*, vol. 66, no. 1, pp. 123-150, 2026.

- [5] M. M. Islam Jim, "Cloud security posture management: Automating risk identification and response in cloud infrastructures," *Academic Journal on Science, Technology, Engineering & Mathematics Education*, vol. 4, no. 3, pp. 10–69593, 2024.
- [6] Bridgecrew, "Checkov: Static code analysis tool for infrastructure as code," 2024. [Online]. Available: <https://www.checkov.io/>
- [7] Accurics, "Terrascan: Detect compliance and security violations across IaC," 2024. [Online]. Available: <https://www.tenable.com/cloud-security/solutions/iac>
- [8] HashiCorp, "Terraform: Infrastructure as code software tool," 2024. [Online]. Available: <https://developer.hashicorp.com/terraform>
- [9] S. Bharti, "Risk assessment and mitigation strategies in cloud systems," 2023.
- [10] K. Wood and E. Pereira, "Impact of misconfiguration in cloud—Investigation into security challenges," *International Journal Multimedia and Image Processing*, vol. 1, no. 1, pp. 17–25, 2011.
- [11] J. Guffey and Y. Li, "Cloud service misconfigurations: Emerging threats, enterprise data breaches and solutions," in *Proc. IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC)*, 2023, pp. 0806–0812.
- [12] T. van Ede, N. Khasuntsev, B. Steen, and A. Continella, "Detecting anomalous misconfigurations in AWS identity and access management policies," in *Proc. Cloud Computing Security Workshop*, 2022, pp. 63–74.
- [13] D. Saxena, I. Gupta, R. Gupta, A. K. Singh, and X. Wen, "An AI-driven VM threat prediction model for multi-risk analysis-based cloud cybersecurity," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 11, pp. 6815–6827, 2023.
- [14] H. Sharma, "The evolution of cybersecurity challenges and mitigation strategies in cloud computing systems," *International Journal of Computer Engineering and Technology*, vol. 15, no. 4, pp. 118–127, 2024.
- [15] I. A. Essien, E. Cadet, J. O. Ajayi, E. D. Erigha, and E. Obuse, "Secure configuration baseline and vulnerability management protocol for multi-cloud environments in regulated sectors," *International Journal of Multidisciplinary Research and Growth Evaluation*, vol. 2, no. 3, pp. 686–696, 2021.
- [16] M. S. Krishnappa et al., "Cybersecurity in the cloud era: Protecting virtualized environments against evolving threats," in *Proc. Int. Conf. Intelligent Cybernetics Technology & Applications (ICICyTA)*, 2024, pp. 1049–1054.
- [17] M. A. Ghorab and M. A. Saied, "Towards secure cloud-native computing: Unveiling Kubernetes misconfigurations with large language models," in *Proc. IEEE Int. Conf. Cloud Computing (CLOUD)*, 2025, pp. 86–96.
- [18] C. Nobles, "Investigating cloud computing misconfiguration errors using the human factors analysis and classification system," *Scientific Bulletin*, vol. 27, no. 1, pp. 59–66, 2022.
- [19] M. K. Goyal and R. Chaturvedi, "Detecting cloud misconfigurations with RAG and intelligent agents: A natural language understanding approach," *SSRN*, 2025.
- [20] D. Kovvuru et al., "A comprehensive study on misconfiguration—SaaS security threat," in *Proc. 2nd Int. Conf. Inventive Computing and Informatics (ICICI)*, 2024, pp. 433–438.